Gym Management System: Architecture and Design

Software Engineering team project · Sawyer Hess, Savvas Christou, Elif Akcivi, Chinguun · October 14, 2026

Project Overview and Scope

The Gym Management System is a React and TypeScript single-page app where members track their membership and their lifts, and staff manage members and check-ins. This document covers the architecture milestone. The section "Prototype Scope" states what the team is coding for this milestone.

- Member: signs up, logs in, views membership status, logs lift sets, views lift history, stats and personal records.
- Staff: logs in, searches and edits members, assigns membership plans, records check-ins.

Out of scope: purchasing or paying for memberships, email verification, password reset by email, multiple gym locations. A real backend is deferred to the week of Nov 30 (ADR-002); until then data lives in the browser. Repository: github.com/savvaschristou/gym-management-system. Final due date: Dec 12, 2026.

Architecture Pattern

Decision: a four-layer architecture inside a client-side React single-page app, with the Repository pattern at the data boundary. Each layer depends only on the layers below it, and storage sits behind repository interfaces, so it can change without touching business rules or UI.

Layer	Folder	Responsibility	May depend on
Presentation	src/ui	Pages, components, routing, route guards. No business rules.	services, domain types
Service	src/services	Use cases: sign in (AuthSession), sign up (SignUpService), later check-in and lift logging. Holds the auth strategies.	domain, data interfaces
Domain	src/domain	Entity types for all seven tables, roles and role home pages; later, rules such as membership status and one-rep max	nothing
Data	src/data	UserRepository and a generic LocalStorageRepository; later an API client	domain

Services receive their repositories through their constructors. App.js creates the concrete classes once at startup and passes them to the pages, so pages never touch localStorage directly.

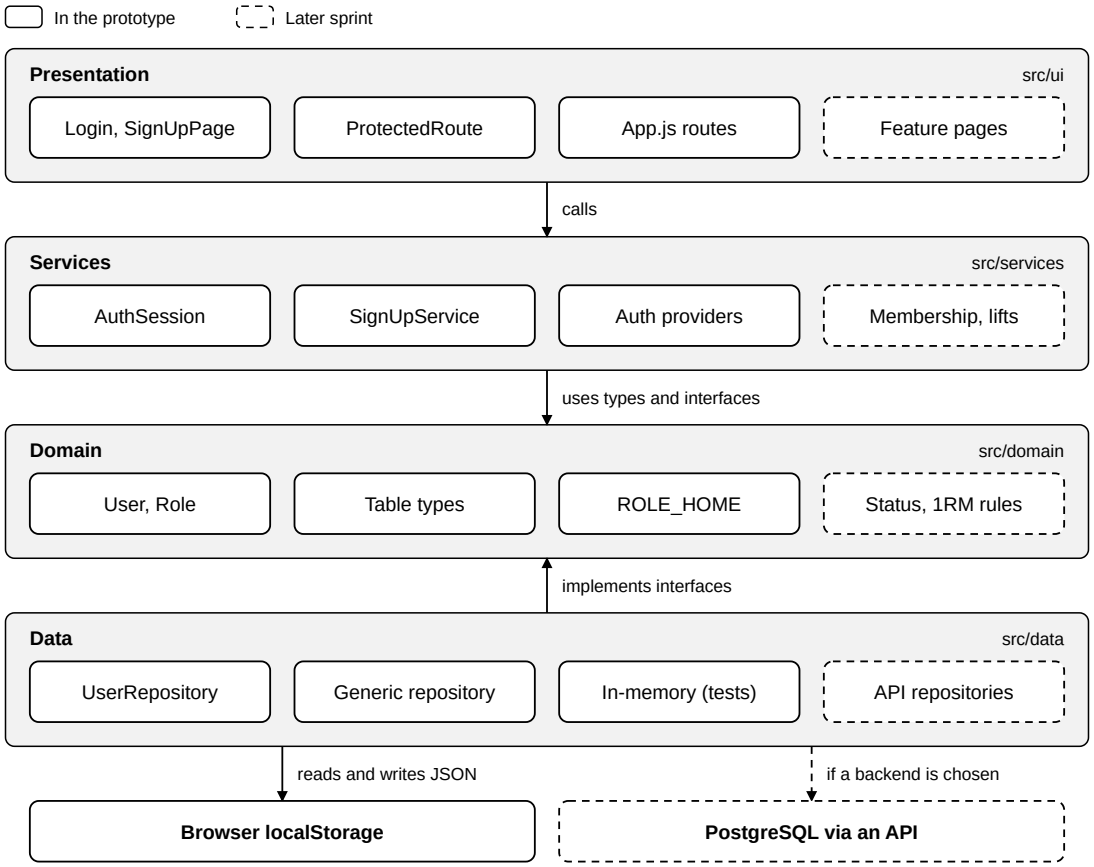
Why this pattern fits:

- It matches the folder layout and SOLID work the team already built, so nothing is restructured.
- The backend decision is deferred. Repositories let the team swap localStorage for an API by writing new data classes, with no change to services or pages.
- Domain and service code is plain TypeScript, testable with Jest without rendering React.
- With four people and about eight weeks left, layers give clean seams for splitting work by feature.

Alternative considered	Why not chosen
MVC	React components blur view and controller; forcing controllers adds ceremony without benefit.
Client-server with a REST backend now	Adds hosting, a second codebase and API work before any features exist.
Microservices	Far too much deployment overhead for a four-person course project.
Feature-sliced folders	Reasonable, but would restructure code already written for the patterns assignment.

Design pattern	Where	Purpose
Repository	UserRepository, LocalStorageRepository in src/data	Hide storage behind interfaces; swap storage later
Strategy	AuthProvider (Local, Demo); later one-rep-max formulas	Swap algorithms without changing callers
Observer	AuthSession.subscribe for sign-in changes; later personal-record listeners	React to events without coupling
Dependency injection	Constructors, wired in App.js	Swap real classes for test doubles

Component Diagram



Arrows show dependencies: the UI calls services, services use domain types and interfaces, and the data layer implements those interfaces. Solid boxes are built in the prototype; dashed boxes arrive in later sprints.

Technology Stack

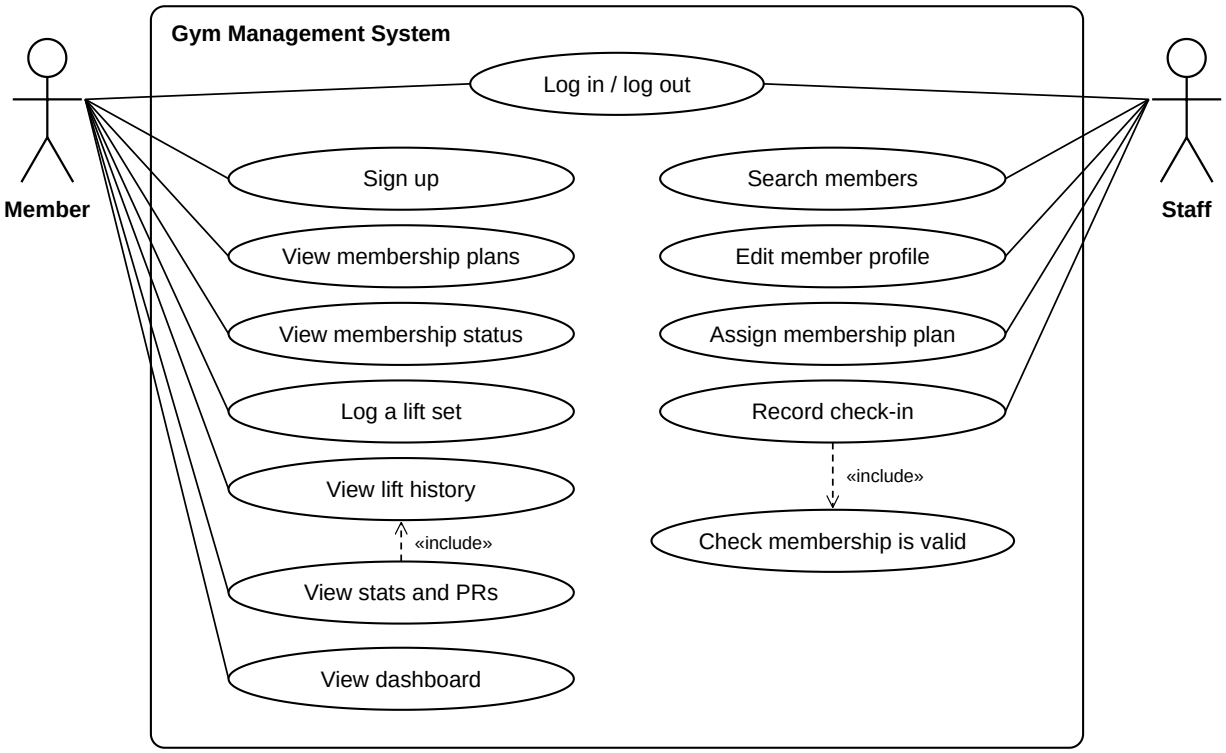
Area	Choice	Notes
Language	TypeScript (strict mode)	Interfaces for repositories and strategies
UI framework	React, scaffolded with Create React App	Already set up in the repo
Routing	React Router	Protected routes by role (member, staff)
State	AuthSession for the signed-in user; local component state elsewhere	No Redux needed at this size

Persistence (prototype)	Browser localStorage behind repositories	JSON per table key, e.g. gym.exercises
Persistence (target)	PostgreSQL, schema in this document	Decided in the week of Nov 30 (ADR-002)
Password hashing	SHA-256 through Web Crypto	Fine for a class demo; a real backend adds per-user salts
Testing	Jest and React Testing Library (bundled with CRA)	Unit tests for domain and services, a few UI tests
Diagrams	Mermaid	Text-based diagrams, kept with the project
Process and tools	Scrum, GitHub Projects, feature branches with pull requests	WhatsApp for team chat

Create React App is deprecated by the React team. It still builds and runs, so the team is not switching mid-semester; a Vite migration is the natural first step if the app outlives the class.

Use Case Diagram

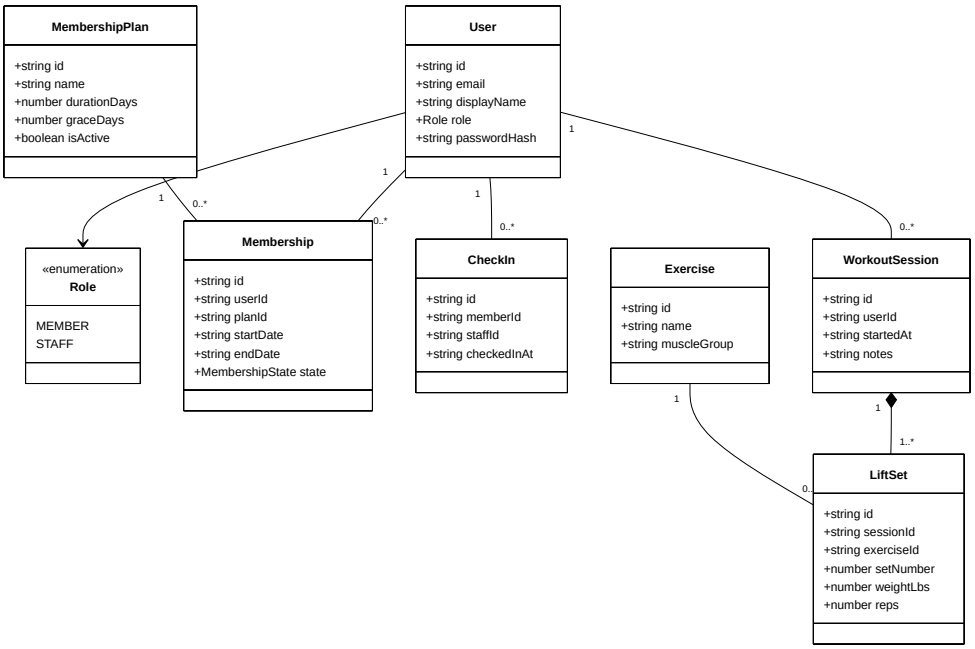
Two actors share authentication; members own their lift data and staff own member administration. Actors are drawn as boxes and use cases as ovals inside the system boundary.



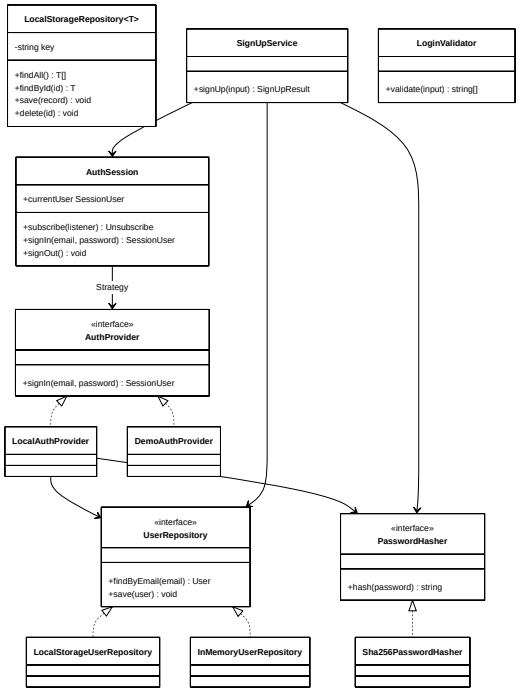
Class Diagram

The diagram is shown in two parts. The domain model mirrors the database tables one to one. The service model shows services depending on repository and strategy interfaces, never on concrete storage (Dependency Inversion).

Domain Model



Service and Infrastructure Model

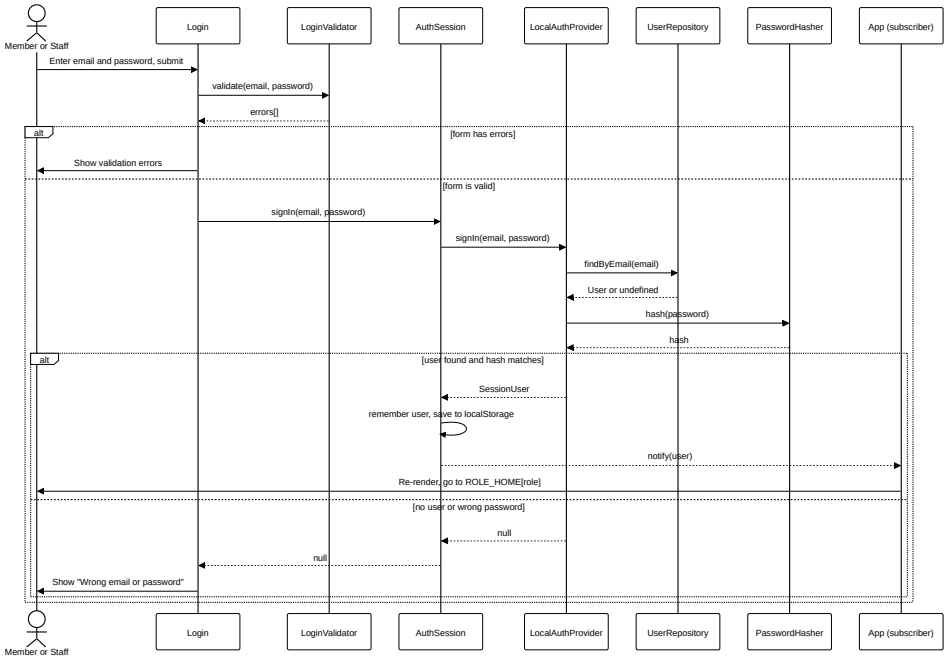


Login, AuthSession, the auth providers and UserRepository were built in the previous sprint. This sprint adds SignUpService, LocalStorageRepository and the types for every table; later sprints add the membership, check-in and lift services.

Sequence Diagrams

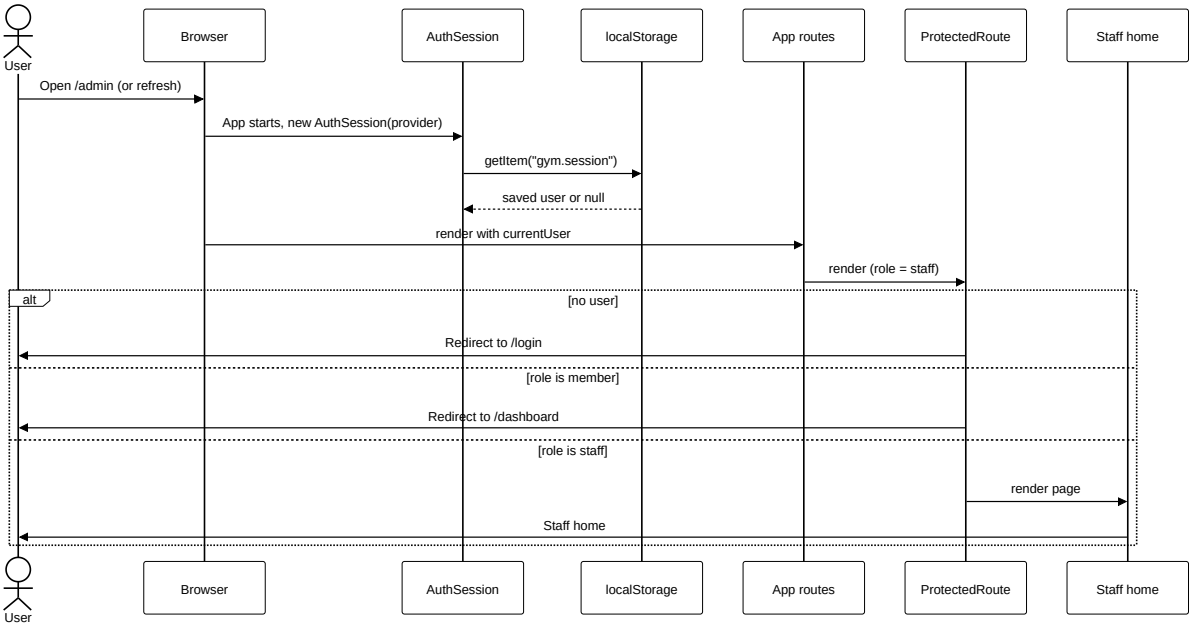
Log In

The Login component never touches storage. AuthSession delegates to the injected AuthProvider (Strategy), then notifies its subscribers, such as App (Observer).



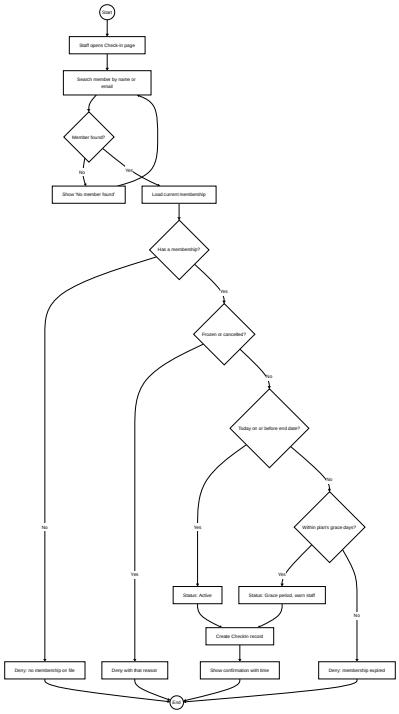
Open a Protected Page

Staying signed in after a refresh and blocking pages by role. AuthSession restores the saved user from localStorage; ProtectedRoute decides what to show.



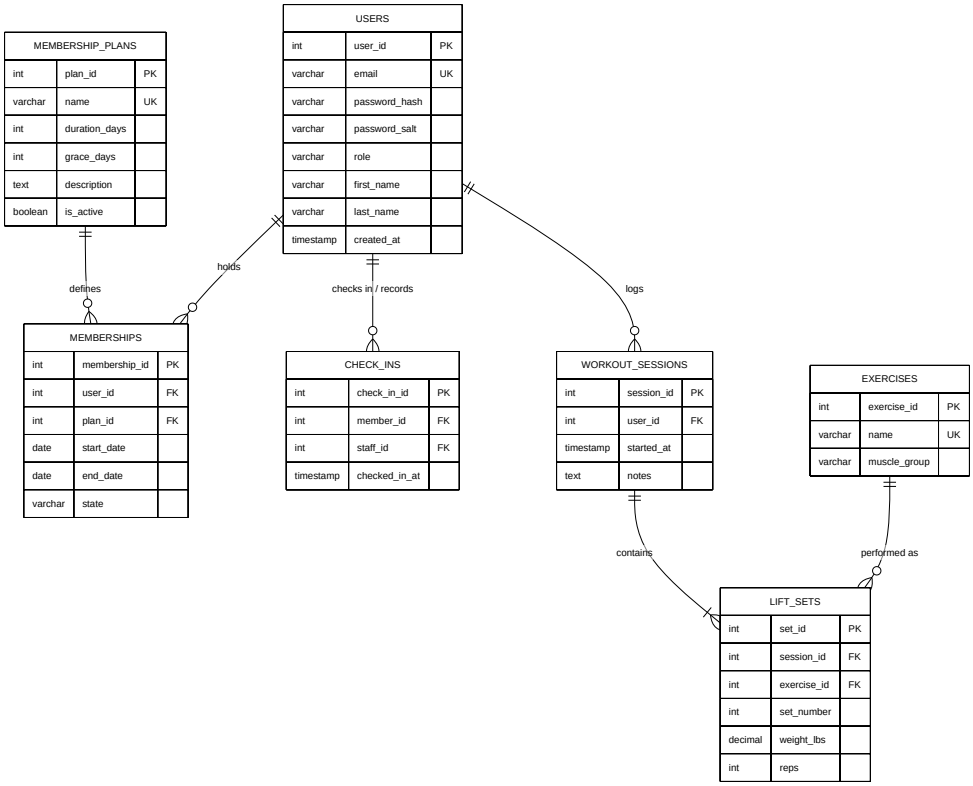
Activity Diagram

Staff check-in: a member is let in only when their membership is active or inside the plan's grace days. This flow is built with the staff tools in the week of Oct 26; the status decision will live in one domain function so every screen agrees.



ER Diagram

Seven tables. Users sit at the center, and lift data hangs off workout sessions rather than off users directly.



Normalized Schema (3NF)

Every table is in third normal form: each non-key column depends on the key, the whole key, and nothing but the key.

- 1NF: every column holds one value. Sets are rows in lift_sets, not a list inside a session.
- 2NF: every table has a single-column surrogate key, so there are no partial dependencies.
- 3NF: no column depends on another non-key column. grace_days and duration_days live in membership_plans, not memberships. muscle_group lives in exercises, not lift_sets. lift_sets has no user_id, because the user is reached through session_id.

Derived values are computed, not stored (ADR-003): expired or grace status comes from end_date, grace_days and today's date; one-rep max estimates, personal records, weekly totals and streaks are calculated from lift_sets. One deliberate allowance: memberships.end_date is stored even though it could be computed from the plan, because staff can extend a membership by hand.

DDL Scripts

The full PostgreSQL schema (all seven tables, constraints and indexes) and the demo data are in the repository at db/schema.sql. The excerpt below shows the pattern every table follows: surrogate keys, foreign keys, and CHECK constraints for allowed values.

```
CREATE TABLE memberships (
    membership_id  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    user_id        INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    plan_id        INTEGER NOT NULL REFERENCES membership_plans(plan_id),
    start_date     DATE NOT NULL,
    end_date       DATE NOT NULL,
    state          VARCHAR(10) NOT NULL DEFAULT 'active'
                   CHECK (state IN ('active', 'frozen', 'cancelled')),
    CHECK (end_date >= start_date)
);

CREATE TABLE lift_sets (
    set_id         INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    session_id     INTEGER NOT NULL REFERENCES workout_sessions(session_id) ON DELETE CASCADE,
    exercise_id    INTEGER NOT NULL REFERENCES exercises(exercise_id),
    set_number     INTEGER NOT NULL CHECK (set_number > 0),
    weight_lbs     NUMERIC(6,2) NOT NULL CHECK (weight_lbs >= 0),
    reps           INTEGER NOT NULL CHECK (reps > 0),
    UNIQUE (session_id, exercise_id, set_number)
);
```

Architecture Decision Records

Four decisions, each Accepted as of Oct 14, 2026. All four are also in the repository at docs/adr.md.

ADR-001: Layered Architecture with Repositories in a Client-Side App

Context: Four students have about eight weeks to build member, staff and lift-tracking features. The backend is undecided, and the team already set up src/domain, src/data, src/services and src/ui.

Decision: Use four layers. Pages call services, services use domain types, and storage hides behind repositories. App.js creates the concrete classes once; pages never touch localStorage.

Consequences: Storage can change without touching UI or services, and domain logic is unit-testable. The cost is a few more files than a quick prototype would need.

ADR-002: localStorage Persistence Until the Backend Decision

Context: Standing up a server and database now would cost a sprint of setup before any feature exists.

Decision: Repositories persist to localStorage, one JSON key per table; the demo accounts are created on startup. The team decides in the week of Nov 30 whether to move to PostgreSQL behind an API.

Consequences: Zero hosting and instant setup. Data is per-browser, can be cleared, and is not secure, so demo accounts are seeded. Moving to a server later means new repository classes, not rewritten features. Alternatives: Firebase or Supabase (couples the domain to a vendor SDK); IndexedDB (harder API than needed).

ADR-003: Store Facts, Compute Derived Values

Context: Membership status, personal records, one-rep max estimates and streaks can all be calculated from other data. Storing them invites values that disagree with their source.

Decision: Persist only facts a person enters or decides. Compute expired and grace status, PRs, 1RM estimates and streaks in domain functions. Observer listeners announce a new PR when a set is logged, but nothing is written for it.

Consequences: The schema stays in 3NF and cannot contradict itself. Stats are recalculated each time, which is trivial at this scale. Alternative: a personal_records table (faster reads, but breaks when a set is edited or deleted).

ADR-004: Role Checks in the Routes, Then in the Services

Context: Members and staff see different pages. A UI guard is easy to bypass, and once there is an API the services become the real boundary.

Decision: This sprint, ProtectedRoute redirects anyone with the wrong role. When staff tools are built (week of Oct 26), each staff-only service method also checks the role, so the rule holds even if a page is bypassed.

Consequences: Once staff tools exist the rule lives in two places, so both change together. Alternatives: UI-only guards; a permissions table (overkill for two roles).

Prototype Scope

The prototype is one thin slice through every layer: sign up, stay signed in after a refresh, and role-protected pages. It is the same work as this week's sprint (Oct 12 to 16), so nothing built for the milestone is thrown away. The work is ten new files plus small changes to four existing ones, with one owner per file.

Owner	Files	What they do
Savvas Christou	AuthSession.ts and its tests, App.js, index.js (existing files)	Save the signed-in user to localStorage and restore it on startup; switch to the local login so new accounts work; create the demo accounts; define the routes
Elif Akcivi	SignUpService.ts and its tests	Validate the form, reject a taken email, save the new member with a hashed password, sign them in
Chinguun	SignUpPage.tsx, ProtectedRoute.tsx and its tests	The sign-up form; the guard that sends signed-out users to /login and wrong-role users to their own home
Sawyer Hess	types.ts, LocalStorageRepository.ts, db/schema.sql, docs/adr.md	Types for all seven tables; one reusable class that saves any table to the browser; the database schema and the decision records

Tests: SignUpService creates a member, rejects a duplicate email, a short password and mismatched passwords, and stores only a hash; ProtectedRoute handles a signed-out user, a wrong-role user and a correct-role user; AuthSession restores, saves and clears the signed-in user.

Demo script:

1. Load the app in a fresh browser.
2. Sign up as a new member and land on /dashboard.
3. Refresh the page; the user is still signed in.
4. Open /admin from the address bar; the member is sent back to /dashboard.
5. Log out, log in as staff@demo.gym, and land on /admin.
6. Run npm test; every test passes.

Deferred to their planned weeks: membership and profile screens, staff search and check-in, service-level role checks, lift logging, stats, the dashboard, styling and any backend. The submitted commit is tagged v0.2-architecture.

Updated Project Plan

The project is on schedule: login is done, and this week's sign up and routing work doubles as the architecture prototype.

Week	Focus	Status	Change since last plan
Oct 5–9	Login with auth Strategy, user Repository, sign-in Observer; design patterns and SOLID assignment	Done	None
Oct 12–16	Sign up, stay signed in, role-protected routes; architecture deliverables; prototype tagged	In progress	Added a generic repository and types for all tables, ADRs, schema file
Oct 19–23	Membership plans and member profiles	Not started	None
Oct 26–30	Staff tools: member list, search, edit, check-in	Not started	Role checks inside staff services (ADR-004)
Nov 2–6	Lift logging: log sets, history per exercise	Not started	None
Nov 9–13	Lift stats: one-rep max (Strategy), PRs and streaks (Observer)	Not started	PRs computed, not stored (ADR-003)
Nov 16–20	Member dashboard: weekly totals, PRs, membership status	Not started	None
Nov 23–27	UI polish, responsive styling, accessibility (Thanksgiving)	Not started	None
Nov 30–Dec 4	Full testing, bug fixes, backend decision, README and docs	Not started	Backend decision framed by ADR-002
Dec 7–11	Final sprint: demo prep, presentation, final write-up	Not started	None
Dec 12	Project due	Not started	None